OMNIS

the partly omniscient auditor

Automated Compliance Evidence Collection & Audit

Detailed Documentation

Architecture, algorithms, results, and design decisions

OMNIS reads security policies into requirements, links each requirement to its evidence, tracks freshness and confidence, detects anomalies, and produces an auditor-ready report with one honest number at the top. This document is the full design writeup: how every stage works, what it measures, and why it was built this way.

Problem statement	PS3, Société Générale iHACK MYPLACE
Author	Ansh Jain (github.com/ataylus)
Team	Partly Everything
Repository	https://github.com/ataylus/OMNIS
Runtime	Python 3.10+, offline, no API key required

What is inside

Full contents on the next page.

Three measured results. Detection precision 0.957 / recall 0.953 on the labelled bench. Evidence linking recovers the correct requirement 54.7% of the time with the ID hidden, against a 6.7% random baseline. The full pipeline runs 500 requirements and 5,000 evidence records in under one second. 102 tests pass.

Contents

1 Abstract

A bank has to prove it follows its own rules. Not just follow them, prove it, to an auditor who accepts nothing less than the receipt: the config file, the rotation log, the access report. Those receipts are scattered across systems, and a compliance team spends days per audit cycle hunting them down. A control can be working perfectly and still fail the audit because nobody found the paperwork.

OMNIS reads the policies, links each requirement to its evidence, tracks how fresh that evidence is, detects anomalies in the evidence corpus, and produces an auditor-ready report with one honest number at the top: the Omniscience Index, a 0 to 100 measure of how much the system actually knows and how much of that it trusts. The guiding idea is that the system quantifies its own ignorance. A requirement with no evidence is reported as UNKNOWN, not quietly skipped. Evidence that matches nothing is left unmapped, not forced.

Every number in this document is reproducible from the repository with a single command and no network access.

2 The problem and the thesis

The four edge cases the brief calls out, missing evidence, conflicting evidence, low-confidence evidence, and ambiguity, are not corner cases bolted on at the end. They are the centre of the problem. Real audit data is incomplete and contradictory, so a tool that only works on clean data is useless. OMNIS treats imperfect data as the job.

Our thesis is honesty as a feature. Most compliance dashboards report a single green number and hide the gaps. OMNIS does the opposite: it surfaces coverage, freshness, and confidence separately, names every gap, and refuses to assert a link or a verdict the data does not support. The headline test of this thesis came from the provided dataset itself, described in Section 5.

3 Architecture

OMNIS is a modular monolith under the omnis/ package. Each stage is a small, testable module that passes typed domain objects to the next. There is no hidden state and no service to stand up: the whole pipeline is a function from files to a report.

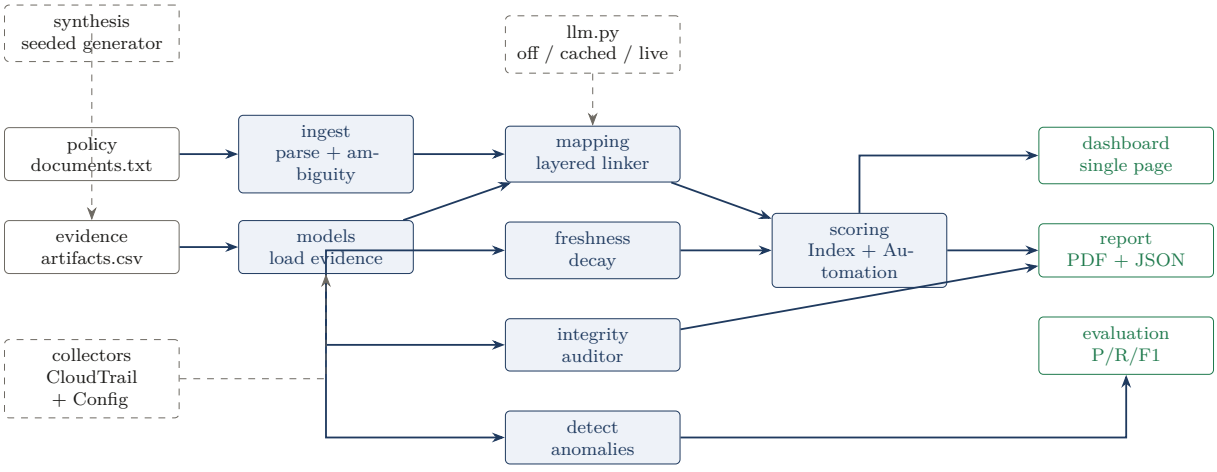


Figure 1: The pipeline. Solid arrows are the core data flow; dashed nodes are optional inputs (synthetic data, mock collectors) and the dormant LLM adapter.

ingest
A deterministic structural parser. Reads the line-structured policy file into typed Requirement

objects, tolerant of the messiness the sample actually contains (CRLF endings, a lowercase scope: field). It also flags ambiguous requirements at parse time (Section 6).

integrity
An evidence-corpus auditor. Eight checks for the defect classes the data contains: duplicate descriptions, orphan references, impossible date orders, freshness that disagrees with the dates, status that contradicts confidence.

mapping
The layered linker (Section 4.1).

freshness
A decay model. Each requirement's audit frequency sets its staleness window; evidence loses weight as it ages past that window.

scoring
Per-requirement status and the headline roll-ups (Section 4.2).

detect
A transparent rule-based anomaly classifier for evidence records, targeting the five ground-truth classes.

narrative
An auditor-voice explanation for each verdict, template-driven by default with an optional LLM enrichment path.

report
The auditor-ready PDF and JSON (Section 10).

evaluation
Precision, recall, and per-class F1 against the ground truth, plus the linking-accuracy metric.

collectors
Two mock integrations that pull evidence into the pipeline (Section 9).

4 The algorithms

4.1 Evidence linking

Linking each evidence record to a requirement runs in layers, stopping at the first confident match. Every link carries the method that produced it and a confidence.

1. **exact_id**: the record's requirement_id equals a known requirement. Confidence 1.0.
2. **framework_rule**: a deterministic match on the record's framework and evidence type against the requirement's compliance mappings and evidence source.
3. **tfidf**: offline TF-IDF cosine similarity between the evidence text and the requirement text, above a similarity floor.
4. **unmapped**: nothing cleared the floor. The record is left unmapped, which is a reported finding, not an error.

The TF-IDF layer is a small pure-Python implementation, so the linker runs offline with no extra dependency. The similarity floor is set to 0.30 deliberately: evidence whose text genuinely matches a requirement is linked, while generic or orphan-referencing evidence is left unmapped

rather than forced onto the nearest requirement. Leaving weak evidence unmapped is the honest outcome and is exactly what the project's thesis demands.

4.2 Compliance scoring and the Omniscience Index

Each requirement is scored from its mapped evidence. A record is *good* when it is approved, fresh, and confident; *failing* when it is rejected, stale without approval, or contradicted (approved on paper but carrying confidence below 0.6); and *ambiguous* otherwise (for example pending review). The status follows:

Status	Condition
COMPLIANT	at least one good record, none failing
GAP	failing records, no good record to offset them
PARTIAL	a mix of good and failing, or only ambiguous evidence
UNKNOWN	no mapped evidence at all (the missing-evidence case)

The Omniscience Index is the mean over requirements of a weighted composite, each component in $[0, 1]$:

$$\text{Index} = \frac{100}{N} \sum_{r=1}^{N} \left(0.4\, c_r + 0.3\, f_r + 0.3\, q_r\right),$$

where c_r is coverage (1 if the requirement has any mapped evidence, else 0), f_r is the best freshness score among its evidence, and q_r is the best confidence. Coverage carries the highest weight because a requirement with no evidence cannot pass an audit at all. Because f_r and q_r are zero when a requirement is uncovered, the Index degrades gracefully toward zero as coverage drops, rather than hiding gaps behind a high average. The Automation Rate is the share of evidence whose type is machine-collectable.

5 We audited the benchmark, and the benchmark failed

The provided dataset has a column, anomaly_marker, that is supposed to be the ground truth for which records are anomalous. Before building a detector against it, we tested it.

> It is noise. A permutation test against every feature in the data shows the labels are no more predictable than chance: best single-threshold precision 0.333, permutation p-value 0.86. The answer key was filled in at random.

Most teams will build a detector, score it against this column, and either overfit quietly or report confusing numbers without knowing why. We did three things instead. We shipped the finding as a reproducible script (make analyze). We built a synthetic bench whose labels come from the data by construction, with 5% injected noise so a perfect score is impossible. And we wired a --labels flag through the evaluator, so the moment the organizers release real ground truth, OMNIS re-grades itself in one command. We set out to build a compliance auditor. The first thing it audited was the thing meant to grade it.

6 The four edge cases

The brief asks specifically how the tool handles ambiguity, conflicting evidence, low-confidence evidence, and missing evidence. Each has a named path and a visible surface.

Missing evidence
becomes UNKNOWN with a plain rationale that the requirement is unproven. It pulls the Index down through the coverage term rather than being skipped.

Conflicting evidence
resolves to PARTIAL, and the narrative writes out the good, failing, and ambiguous split so the reasoning is visible.

Low-confidence evidence
is caught two ways: the integrity auditor raises a status-confidence contradiction for approved records carrying confidence below 0.6, and the scorer counts such a record against its requirement rather than for it.

Ambiguous requirements
are flagged at parse time. A requirement that states a principle with no measurable threshold ("principle of least privilege", "no personal use") has no objective pass or fail, unlike "encrypted with AES-256" or "retained for 90 days". OMNIS flags these so the auditor knows to apply judgment. Ambiguous *evidence* (present but inconclusive) is handled separately as PARTIAL.

7 Results

All numbers are measured offline with the LLM off, reproducible from the repository.

Measure	Result	Bar
Anomaly detection (synthetic bench)	P 0.957 / R 0.953 / F1 0.955	P > 0.70, R > 0.60
Evidence linking, exact requirement (ID hidden)	54.7%	6.7% random
Evidence linking, correct policy area	60.7%	6.7% random
Omniscience Index (synthetic / sample)	92.4 / 64.4	—
Automation Rate (synthetic / sample)	78.0% / 48.6%	70%+ target
Full pipeline, 500 requirements + 5,000 evidence	under 1 second	under 60 seconds
Test suite	102 passing	—

Table 1: Headline measurements. The detection score is on the synthetic bench, whose labels are constructed; the provided sample is kept as the noisy real-world test.

A note on honesty, since it matters here. The synthetic detection score would be circular on its own, because the bench is labelled by the same rule logic the detector uses. That is exactly why we add injected noise so a perfect score is impossible, keep the noisy provided sample as the primary real-world test, and grade the linker with the ID it relies on hidden. We would rather report a measured 54.7% than a circular 100%.

8 The data, and the integrity auditor

The provided corpus is deliberately broken, and OMNIS reports each defect rather than crashing. On the 500 sample rows it finds: all 500 sharing one identical requirement description; hundreds of requirement IDs that exist in no policy; 239 rows reviewed before they were collected; 455 rows whose stored freshness disagrees with their own dates by more than a week; and 18 rows marked approved while carrying low confidence. Each is tagged with a severity and reported as a data-quality finding.

The synthetic bench is the coherent counterpart. A seeded, configurable generator (omnis synth) produces an enterprise dataset with ground-truth labels by construction across six policies and fifteen requirements, documented in data/synthetic/DATA_CARD.md.

9 Evidence collectors

Auto-collection is the first build goal of the brief. OMNIS ships two collectors as mock integrations: a CloudTrail-style log puller and a config-snapshot puller. They are mocked in one

specific way, each reads a committed sample export instead of calling a live API. Everything else is real: they emit the same EvidenceRecord objects the pipeline consumes, and omnis collect runs them end to end, pulling nine automated records that link to requirements by exact ID. Swapping the file read for a live API call is the single remaining integration step.

10 User interface

OMNIS presents results two ways: an interactive dashboard and a typeset report.

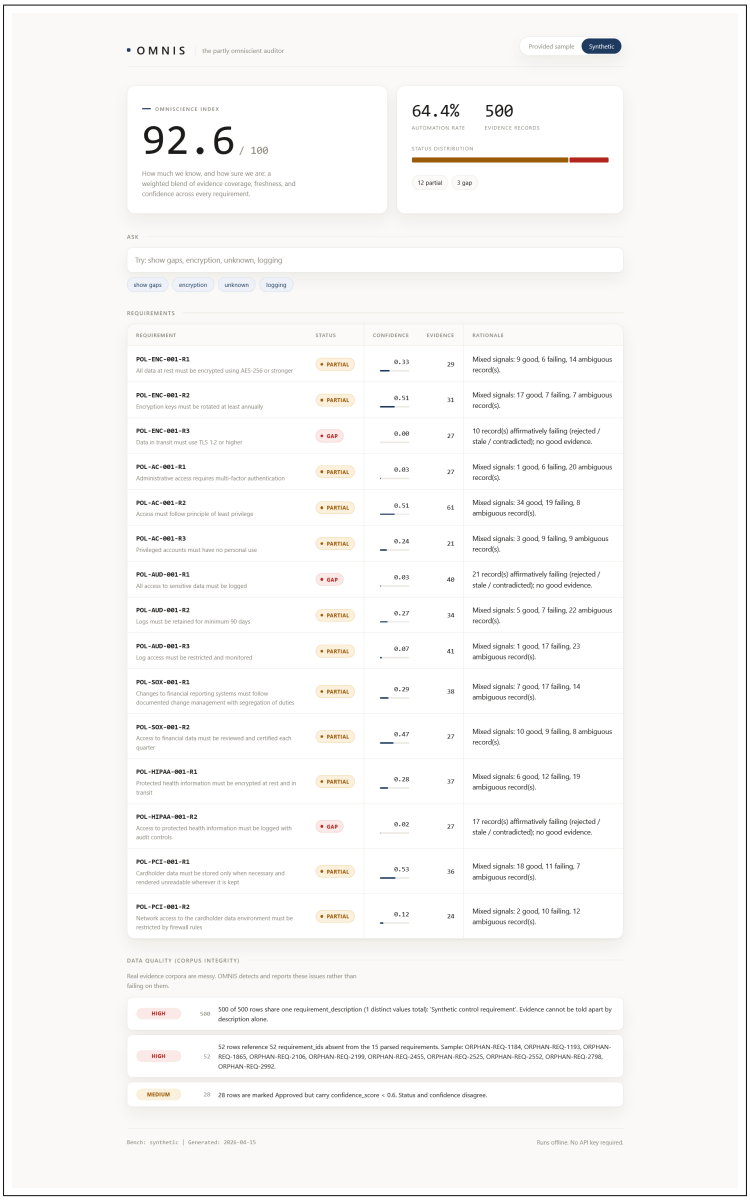
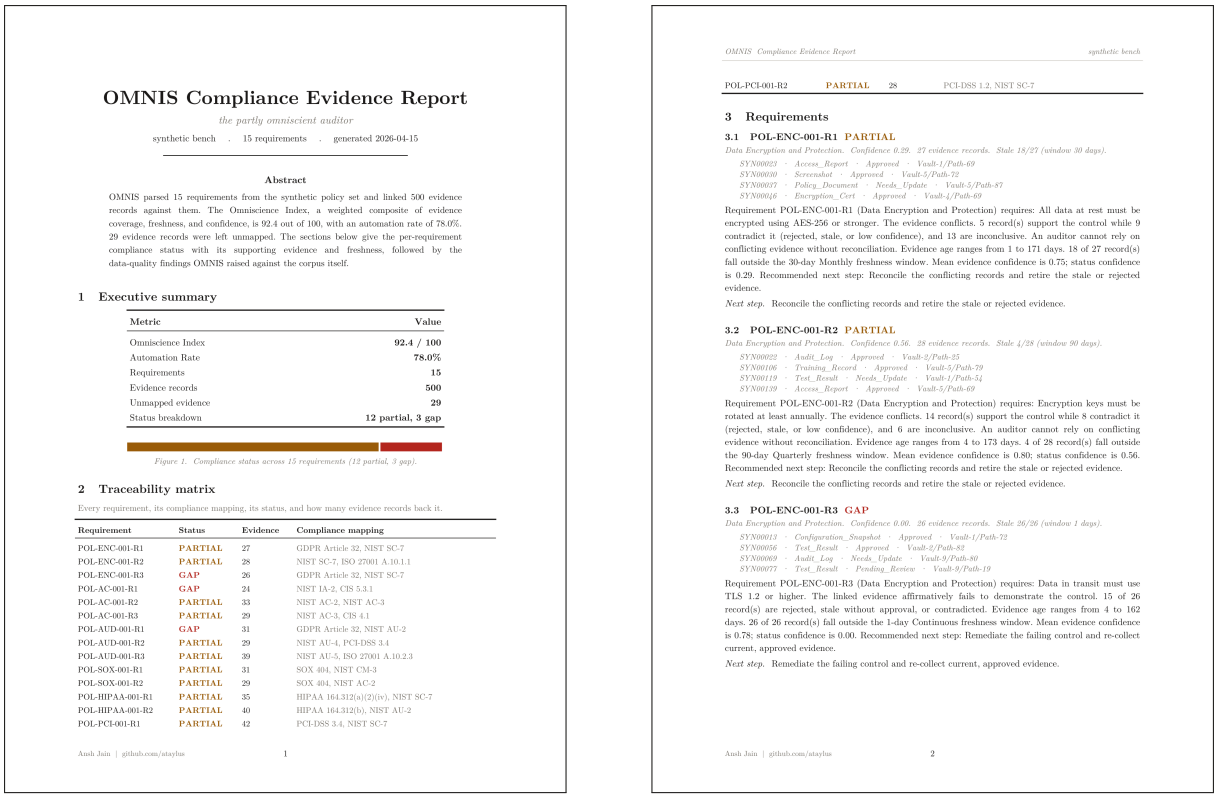


Figure 2: The single-page dashboard. Omniscience Index hero, automation rate, a status distribution bar, the requirement table with status chips, and a data-quality panel. It is one HTML file served by the Python standard library, no framework and no build step.



OMNIS Compliance Evidence Report

the partly omniscient auditor

synthetic bench . 15 requirements . generated 2026-04-15

Abstract

OMNIS parsed 15 requirements from the synthetic policy set and linked 500 evidence records against them. The Omniscience Index, a weighted composite of evidence coverage, freshness, and confidence, is 92.4 out of 100, with an automation rate of 78.0%. 29 evidence records were left unmapped. The sections below give the per-requirement compliance status with its supporting evidence and freshness, followed by the data-quality findings OMNIS raised against the corpus itself.

1 Executive summary

Metric	Value
Omniscience Index	**92.4 / 100**
Automation Rate	**78.0%**
Requirements	**15**
Evidence records	**500**
Unmapped evidence	**29**
Status breakdown	**12 partial, 3 gap**

Figure 1. Compliance status across 15 requirements (12 partial, 3 gap).

2 Traceability matrix

Every requirement, its compliance mapping, its status, and how many evidence records back it.

Requirement	Status	Evidence	Compliance mapping
POL-ENC-001-R1	PARTIAL	27	GDPR Article 32, NIST SC-7
POL-ENC-001-R2	PARTIAL	28	NIST SC-7, ISO 27001 A.10.1.1
POL-ENC-001-R3	GAP	26	GDPR Article 32, NIST SC-7
POL-AC-001-R1	GAP	24	NIST IA-2, CIS 5.3.1
POL-AC-001-R2	PARTIAL	33	NIST AC-2, NIST AC-3
POL-AC-001-R3	PARTIAL	29	NIST AC-3, CIS 4.1
POL-AUD-001-R1	GAP	31	GDPR Article 32, NIST AU-2
POL-AUD-001-R2	PARTIAL	29	NIST AU-4, PCI-DSS 3.4
POL-AUD-001-R3	PARTIAL	39	NIST AU-5, ISO 27001 A.10.2.3
POL-SOX-001-R1	PARTIAL	31	SOX 404, NIST CM-3
POL-SOX-001-R2	PARTIAL	29	SOX 404, NIST AC-2
POL-HIPAA-001-R1	PARTIAL	35	HIPAA 164.312(a)(2)(iv), NIST SC-7
POL-HIPAA-001-R2	PARTIAL	40	HIPAA 164.312(b), NIST AU-2
POL-PCI-001-R1	PARTIAL	42	PCI-DSS 3.4, NIST SC-7

Ansh Jain | github.com/ataylus 1

POL-PCI-001-R2 PARTIAL 28 PCI-DSS 1.2, NIST SC-7

3 Requirements

3.1 POL-ENC-001-R1 PARTIAL

Data Encryption and Protection. Confidence 0.29. 27 evidence records. Stale 18/27 (window 30 days).

SYN00023 · Access_Report · Approved · Vault-1/Path-69
SYN00030 · Screenshot · Approved · Vault-5/Path-72
SYN00037 · Policy_Document · Needs_Update · Vault-5/Path-87
SYN00046 · Encryption_Cert · Approved · Vault-4/Path-69

Requirement POL-ENC-001-R1 (Data Encryption and Protection) requires: All data at rest must be encrypted using AES-256 or stronger. The evidence conflicts. 5 record(s) support the control while 9 contradict it (rejected, stale, or low confidence), and 13 are inconclusive. An auditor cannot rely on conflicting evidence without reconciliation. Evidence age ranges from 1 to 171 days. 18 of 27 record(s) fall outside the 30-day Monthly freshness window. Mean evidence confidence is 0.75; status confidence is 0.29. Recommended next step: Reconcile the conflicting records and retire the stale or rejected evidence.

Next step. Reconcile the conflicting records and retire the stale or rejected evidence.

3.2 POL-ENC-001-R2 PARTIAL

Data Encryption and Protection. Confidence 0.56. 28 evidence records. Stale 4/28 (window 90 days).

SYN00022 · Audit_Log · Approved · Vault-2/Path-25
SYN00106 · Training_Record · Approved · Vault-5/Path-79
SYN00119 · Test_Result · Needs_Update · Vault-1/Path-54
SYN00139 · Access_Report · Approved · Vault-5/Path-69

Requirement POL-ENC-001-R2 (Data Encryption and Protection) requires: Encryption keys must be rotated at least annually. The evidence conflicts. 14 record(s) support the control while 8 contradict it (rejected, stale, or low confidence), and 6 are inconclusive. An auditor cannot rely on conflicting evidence without reconciliation. Evidence age ranges from 4 to 173 days. 4 of 28 record(s) fall outside the 90-day Quarterly freshness window. Mean evidence confidence is 0.80; status confidence is 0.56. Recommended next step: Reconcile the conflicting records and retire the stale or rejected evidence.

Next step. Reconcile the conflicting records and retire the stale or rejected evidence.

3.3 POL-ENC-001-R3 GAP

Data Encryption and Protection. Confidence 0.00. 26 evidence records. Stale 26/26 (window 1 days).

SYN00013 · Configuration_Snapshot · Approved · Vault-1/Path-72
SYN00056 · Test_Result · Approved · Vault-2/Path-83
SYN00069 · Audit_Log · Needs_Update · Vault-9/Path-80
SYN00077 · Test_Result · Pending_Review · Vault-9/Path-19

Requirement POL-ENC-001-R3 (Data Encryption and Protection) requires: Data in transit must use TLS 1.2 or higher. The linked evidence affirmatively fails to demonstrate the control. 15 of 26 record(s) are rejected, stale without approval, or contradicted. Evidence age ranges from 4 to 162 days. 26 of 26 record(s) fall outside the 1-day Continuous freshness window. Mean evidence confidence is 0.78; status confidence is 0.00. Recommended next step: Remediate the failing control and re-collect current, approved evidence.

Next step. Remediate the failing control and re-collect current, approved evidence.

Ansh Jain | github.com/ataylus 2

Figure 3: The auditor-ready report. A summary with the Omniscience Index, a traceability matrix (requirement, status, evidence count, compliance mapping), then a section per requirement with its linked evidence pointers and narrative, and an integrity appendix.

11 Design decisions

The choices that shaped OMNIS, and the one-line reason behind each.

Decision	Why
Deterministic parser first	Policy structure is structure; you parse it, you do not send it to a model and hope.
Layered linker, loud UNMAPPED	Cheap exact matches first, similarity only as a fallback, and an explicit unmapped verdict when nothing fits.
Rules before ML	A rule you can defend in a sentence beats an opaque model you cannot, especially once the provided labels turned out to be noise.
Recompute, do not trust	The stored freshness field is wrong on 455 of 500 rows, so freshness is recomputed from a fixed reference date, which also makes runs reproducible.
LLM off by default	The whole system runs on templates with no key, no network, and no cost. Live mode is wired and logs its own spend, but nothing depends on it.
Standard-library dashboard	One HTML file, no framework, no build step. It opens and works on a judge's laptop.

Table 2: The decisions that shaped OMNIS, and the reasoning behind each.

12 LLM policy and cost

There is one LLM adapter, omnis/llm.py, with three modes. **off**, the shipped default, returns nothing so every caller falls back to a deterministic template; the whole system runs with no key, no network, and no cost. **cached** replays a recorded response. **live** calls the API only if a key is present and logs every call with its token estimate and cost. The performance numbers are measured with the LLM off; the model never sits on the critical path.

Cost, as an estimate from the adapter's token heuristic and not a bill: a narrative enrichment is about 400 input and 150 output tokens, roughly $0.0012 per requirement, about $0.02 for all fifteen synthetic requirements and about $0.58 even at the full 500-requirement scale. The real spend to date is $0.00, because the system ships with the LLM off.

13 Limitations

Stated plainly. The two collectors are mock integrations: they read sample exports rather than calling live APIs. The content linker is TF-IDF, a solid offline baseline rather than semantic embeddings, so its exact-requirement recovery with the ID hidden is 54.7%, well above random but short of perfect. The synthetic bench is labelled by construction, which is why the noisy provided sample remains the primary real-world test. Performance is measured on a single laptop. None of these are hidden; each is reported where it is relevant.

14 Reproducibility

Everything runs offline on a clean machine:

```
pip install -r requirements.txt
make demo                                  opens the dashboard
make test                                  runs the 102 tests
make analyze                               reproduces the label-independence finding
python -m omnis report --bench synthetic   writes the report
python -m omnis perf                       times 500 requirements + 5,000 evidence
```

The full source, the committed report, and the analysis notebook are in the repository at https://github.com/ataylus/OMNIS.